Yes. This is a very workable system, and I think the key is to design it around **one central service request/invoice record**, regardless of how the customer contacts Willis Port.

From what you described, the application is really doing four connected jobs: **capture customer requests → calculate shipping charges → issue invoices → record payment and issue receipts.**

A good starting workflow would look like this:

Customer contacts Willis Port through Email, WhatsApp, Phone Call, Walk-in, or the Internet/Home website. Staff create a request in the system, or an online customer request creates it automatically. The request records the **source**, so management can later see where business is coming from.

For example:

```
Request #WP-2026-000154
```

Customer: John Mensah
Source: WhatsApp
Service: Shipping
Goods Type: Normal Goods
Weight: 420 kg
Volume: 1.8 CBM
Currency basis: USD
Status: Quote / Invoice Pending

Then Willis Port staff enter the shipment details.

I would separate the shipping information into fields such as:

- Customer name, telephone, WhatsApp number and email
- Request source: Email / WhatsApp / Phone / Walk-in / Website
- Type of shipment: **Air Special / Shipped / other services**
- Goods classification: **Normal Goods / Special Goods**
- Description of goods
- Number of packages
- Total kilograms
- Total CBM
- Supporting image
- Supporting PDF/document
- Date received
- Invoice validity date

- Notes

The files should not merely be saved under the customer's name. Internally I would give every request a permanent unique number and attach the client's name to it.

For example:

WP-2026-000154 — John Mensah

That prevents problems if you have five different customers named John Mensah.

The pricing engine is particularly important

Instead of staff manually calculating every invoice, Willis Port should maintain a **Price List** inside the system.

For example:

Service	Goods Type	Pricing Basis	Rate
Sea Shipping	Normal	CBM	$260 / CBM
Sea Shipping	Special	CBM	configurable
Air Special	Special	KG	configurable
Air Freight	Normal	KG	configurable

I'm interpreting your statement about **260 per CBM** as meaning something such as **US$260 per CBM**. We should confirm that exact business rule before implementing it.

If a shipment were:

2.4 CBM × $260 = $624

the system calculates $624 automatically.

But there is another important requirement you mentioned:

> Currency in dollars — use the market rate — at the end use Ghana cedis to charge.

I recommend storing **both values**.

For example:

USD Charge: **$624.00**

Exchange rate used: **GHS 11.20 / USD**

Final amount payable: **GHS 6,988.80**

Most importantly, once the invoice is generated, **the exchange rate used for that invoice should be frozen**.

Even if tomorrow's exchange rate changes, yesterday's invoice should still show the rate that was applicable when it was issued. If the invoice expires and Willis Port generates a new invoice, the system can use the latest applicable rate.

This gives you a proper audit trail.

Invoice generation

When the calculation is approved, the application creates an invoice number such as:

INV-WP-2026-000154

The invoice PDF could show:

WILLIS PORT

Customer information
Invoice number
Invoice date
Validity / expiry date
Goods description
Shipment method
Goods classification
Weight in KG
Volume in CBM
Rate applied
USD equivalent
USD/GHS exchange rate
Amount payable in Ghana cedis
Payment instructions
Terms and conditions

And because you're using **white, black and orange**, we can make the invoice and application interface follow the same branding.

Something like:

White — primary background
Black — headings/navigation/text
Orange — buttons, totals, important status indicators, Willis Port branding

That can look very professional.

Sending the invoice

After generation, staff should have something like:

Send Invoice

and select:

```
WhatsApp | Email | Download PDF | Print
```

Eventually, WhatsApp and email could be integrated directly so that Willis Port doesn't have to download the invoice and manually attach it every time.

The application should record:

Invoice sent: Yes
Sent through: WhatsApp
Sent at: August 13, 2026 10:41 AM

This becomes useful for the management statistics you mentioned.

Payment and receipt

An invoice should have a clear lifecycle.

Something like:

Draft → Approved → Sent → Partially Paid → Paid

And perhaps:

Expired / Cancelled

When payment is received, staff open the invoice and choose **Record Payment**.

They enter:

Payment amount
Payment method
Reference number
Payment date
Notes

Once the invoice is fully paid, the system generates something like:

RCT-WP-2026-000154

The receipt should reference the original invoice:

Invoice: INV-WP-2026-000154
Amount invoiced: GHS 6,988.80
Amount paid: GHS 6,988.80
Outstanding: GHS 0.00
Status: **PAID**

That is important because **invoice and receipt are different documents**. An invoice asks the customer to pay; the receipt proves that payment was received.

Management dashboard

Your last point—*"Total number of people invoices sent to / money paid or not"*—is where the application becomes more valuable than simply generating PDFs.

The dashboard could immediately show:

Today

Invoices created: **28**
Invoices sent: **25**
Paid: **17**
Unpaid: **8**
Expired: **3**

Amount invoiced: **GHS 143,820**

Amount collected: **GHS 97,500**

Outstanding: **GHS 46,320**

And Willis Port could filter this by:

Today
This week
This month
Customer
Employee
Email
WhatsApp
Phone
Walk-in
Website
Normal goods
Special goods
Air
Sea

This means they will eventually be able to answer questions such as:

"How much business came through WhatsApp this month?"

"How many invoices haven't been paid?"

"What is the value of outstanding invoices?"

"Which customers owe us?"

"How many CBM did we ship this month?"

That's a proper business management system rather than just an invoicing application.

One other design decision I would strongly recommend: **don't make Invoice the starting point of the database.**

Start with:

Customer → Service Request → Shipment/Job → Invoice → Payment → Receipt

One customer can therefore have many requests, and one request can generate an invoice and later a receipt.

Conceptually:

```
CUSTOMER
   |
```

```
    ├── SERVICE REQUEST
    │       │
    │       ├── Shipment details
    │       ├── Images/PDFs
    │       ├── Weight
    │       ├── CBM
    │       └── Pricing
    │
    └──────── INVOICE
                  │
                  └── PAYMENT
                          │
                          └── RECEIPT
```

This structure will serve Willis Port much better as the company grows.

And I already see a natural **Phase 2** later: shipment tracking, customer accounts, WhatsApp notifications, payment links/mobile money, overdue invoice reminders, customer statements, accounting exports and operational reports.

For the first version, though, I would keep the goal very focused:

Capture every enquiry → calculate the correct shipping charge → approve it → generate/send the invoice → record payment → issue the receipt → report what is paid and outstanding.

That's the core Willis Port system I would build first.